

Data Structures — Reference Handbook

Data Structures (DS) · Unit 1 · Arrays, Linked Lists & Complexity · 2024

Overview

These notes cover **Data Structures**. Core data structures and the algorithms built on them — arrays, lists, stacks, trees, graphs, sorting and hashing. Each section states the core idea, a worked intuition and the points examiners look for.

1. Asymptotic Notation

Big-O, Omega and Theta describe upper, lower and tight bounds on an algorithm's growth rate.

Why it matters: Asymptotic Notation appears regularly in mid and semester examinations. Be ready to define it precisely, draw a diagram where relevant, and give one realistic example.

- Key definition of asymptotic notation in one line.
- A small example showing asymptotic notation in practice.
- Common mistakes students make and how to avoid them.

2. Arrays

Arrays store elements contiguously, giving $O(1)$ indexed access but $O(n)$ insertion in the middle.

Why it matters: Arrays appears regularly in mid and semester examinations. Be ready to define it precisely, draw a diagram where relevant, and give one realistic example.

- Key definition of arrays in one line.
- A small example showing arrays in practice.
- Common mistakes students make and how to avoid them.

3. Singly Linked Lists

Each node stores data and a pointer to the next node, allowing $O(1)$ insertion at the head.

Why it matters: Singly Linked Lists appears regularly in mid and semester examinations. Be ready to define it precisely, draw a diagram where relevant, and give one realistic example.

- Key definition of singly linked lists in one line.
- A small example showing singly linked lists in practice.
- Common mistakes students make and how to avoid them.

4. Stack ADT

A stack is a LIFO structure supporting push, pop and peek in constant time.

Why it matters: Stack ADT appears regularly in mid and semester examinations. Be ready to define it precisely, draw a diagram where relevant, and give one realistic example.

- Key definition of stack adt in one line.
- A small example showing stack adt in practice.
- Common mistakes students make and how to avoid them.

5. Infix to Postfix Conversion

Operator precedence and a stack convert infix expressions to postfix for easy evaluation.

Why it matters: Infix to Postfix Conversion appears regularly in mid and semester examinations. Be ready to define it precisely, draw a diagram where relevant, and give one realistic example.

- Key definition of infix to postfix conversion in one line.
- A small example showing infix to postfix conversion in practice.
- Common mistakes students make and how to avoid them.

6. Recursion and the Call Stack

Each recursive call pushes a frame; tail recursion can be converted to iteration.

Why it matters: Recursion and the Call Stack appears regularly in mid and semester examinations. Be ready to define it precisely, draw a diagram where relevant, and give one realistic example.

- Key definition of recursion and the call stack in one line.
- A small example showing recursion and the call stack in practice.
- Common mistakes students make and how to avoid them.

7. Binary Tree Traversals

Inorder, preorder, postorder and level-order traversals visit every node in a defined order.

Why it matters: Binary Tree Traversals appears regularly in mid and semester examinations. Be ready to define it precisely, draw a diagram where relevant, and give one realistic example.

- Key definition of binary tree traversals in one line.
- A small example showing binary tree traversals in practice.
- Common mistakes students make and how to avoid them.

8. Binary Search Trees

In a BST every left descendant is smaller and every right descendant larger, enabling $O(h)$ search.

Why it matters: Binary Search Trees appears regularly in mid and semester examinations. Be ready to define it precisely, draw a diagram where relevant, and give one realistic example.

- Key definition of binary search trees in one line.
- A small example showing binary search trees in practice.
- Common mistakes students make and how to avoid them.

9. AVL Trees

AVL trees keep balance factors in $\{-1, 0, 1\}$ using LL, RR, LR and RL rotations.

Why it matters: AVL Trees appears regularly in mid and semester examinations. Be ready to define it precisely, draw a diagram where relevant, and give one realistic example.

- Key definition of avl trees in one line.
- A small example showing avl trees in practice.
- Common mistakes students make and how to avoid them.

10. Graph Representations

Adjacency matrices use $O(V^2)$ space while adjacency lists use $O(V + E)$.

Why it matters: Graph Representations appears regularly in mid and semester examinations. Be ready to define it precisely, draw a diagram where relevant, and give one realistic example.

- Key definition of graph representations in one line.
- A small example showing graph representations in practice.
- Common mistakes students make and how to avoid them.

11. Breadth-First Search

BFS explores level by level using a queue and finds shortest paths in unweighted graphs.

Why it matters: Breadth-First Search appears regularly in mid and semester examinations. Be ready to define it precisely, draw a diagram where relevant, and give one realistic example.

- Key definition of breadth-first search in one line.
- A small example showing breadth-first search in practice.
- Common mistakes students make and how to avoid them.

12. Depth-First Search

DFS explores as deep as possible using recursion or a stack and underlies topological sort.

Why it matters: Depth-First Search appears regularly in mid and semester examinations. Be ready to define it precisely, draw a diagram where relevant, and give one realistic example.

- Key definition of depth-first search in one line.
- A small example showing depth-first search in practice.
- Common mistakes students make and how to avoid them.

13. Binary Search

Binary search halves a sorted range each step, running in $O(\log n)$.

Why it matters: Binary Search appears regularly in mid and semester examinations. Be ready to define it precisely, draw a diagram where relevant, and give one realistic example.

- Key definition of binary search in one line.
- A small example showing binary search in practice.
- Common mistakes students make and how to avoid them.

14. Quick Sort

Quick sort partitions around a pivot; average $O(n \log n)$, worst case $O(n^2)$.

Why it matters: Quick Sort appears regularly in mid and semester examinations. Be ready to define it precisely, draw a diagram where relevant, and give one realistic example.

- Key definition of quick sort in one line.
- A small example showing quick sort in practice.
- Common mistakes students make and how to avoid them.

15. Merge Sort

Merge sort divides the array, sorts halves recursively and merges them in $O(n \log n)$.

Why it matters: Merge Sort appears regularly in mid and semester examinations. Be ready to define it precisely, draw a diagram where relevant, and give one realistic example.

- Key definition of merge sort in one line.
- A small example showing merge sort in practice.
- Common mistakes students make and how to avoid them.

Quick revision

Asymptotic Notation · Arrays · Singly Linked Lists · Stack ADT · Infix to Postfix Conversion · Recursion and the Call Stack · Binary Tree Traversals · Binary Search Trees · AVL Trees · Graph Representations · Breadth-First Search · Depth-First Search · Binary Search · Quick Sort · Merge Sort